腾讯广泛使用的跨端开发框架——Kuikly在搜狗输入法中的AI Coding实践

腾讯程序员 腾讯技术工程 2026年3月27日 02:35 广东 15人



作者：搜狗输入法 hesleyliang

导语

AI 以超乎预期的速度席卷开发领域，AI 工程化也成为各业务团队优先探讨的命题。输入法团队从实际需求出发，在Kuikly跨端项目中逐步探索并沉淀出一套 AI 工程化方案，目前已有不少需求按此流程完成开发并上线。以此文与大家分享其中的实践经验与思考，希望为同样在 AI Coding 工程化路上探索的团队提供一些可参考的思路。

Kuikly是腾讯广泛使用的跨端开发框架，提供了使用Kotlin语言开发Android、iOS、鸿蒙、Web、小程序、MAC跨端应用能力。目前已在 QQ、腾讯新闻、QQ 音乐、搜狗输入法、QQ 浏览器等20+业务深度使用，服务业务的总页面数1000+、日活用户超5亿，满足了这些业务在众多场景下的各类复杂需求（应用场景案例）

一、背景

在AI时代，各业务团队都在积极探索AI Coding，目标相似——希望实现需求自动关联、代码生成、效果测试一站式的AI愿景，达到“L3级别”的代码生成。网上各类Vibe Coding演示，从0到1搭建Demo应用的过程已经相当流畅，似乎也印证了这一方向的可行性。

输入法Kuikly跨端项目同样面临这样的机遇，作为一个横跨 Android、iOS、鸿蒙、H5 多端的复杂跨平台工程，在跨端的背景下已经大幅提升了开发效率，新页面通常由单人完成多端交付。而在 AI 时代，我们希望更进一步——借助 AI Coding 的能力，将开发效率再提升一个台阶。

从2025年下半年起，团队开始逐步落地实践。辅助问答、文档查询、代码生成等能力确实为开发带来了一定提效，但在面对完整页面需求时，AI生成的代码虽然能运行，但距离真正“可用”仍有差距。

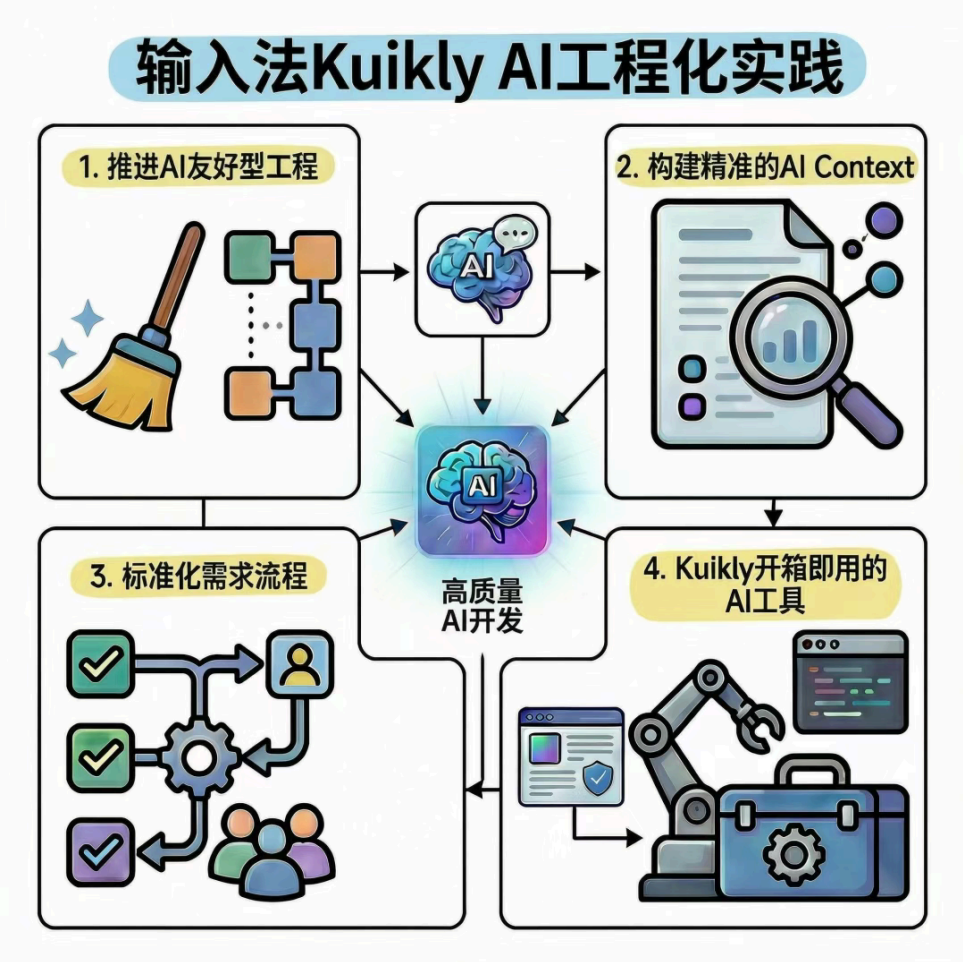
- **存量工程理解有幻觉**。输入法 Kuikly 工程经过多年积累，已封装了大量基础能力，但 AI 对我们工程的理解仍存在明显偏差，常出现 API 调用幻觉，遇到需求时倾向于重复造轮子等问题。此外，对于客户端常见的页面迭代场景，AI 也难有效处理。

● **需求缺乏结构化输入**。Vibe Coding 模式下，需求细节往往依赖开发自己补全，模糊的需求交给 AI，产出的代码同样缺乏准确性。当发现方向偏离时，往往已经经历了多轮修改。需求中未澄清的部分越多，返工成本越高。

基于以上问题，我们思考：要让AI在真实工程中真正发挥作用，实现真正的工程化，需要满足哪些条件？输入法跨端Kuikly项目也在实际的页面开发中探索。

二、AI工程化

在正式探索 Spec coding 之前，我们最初直接以 Vibe Coding 的方式使用 AI，随后接入了 Kuikly 团队提供的 AI 工具，在组件开发和小需求场景中已经有不错的效果。在此基础上，我们希望更进一步，通过更规范的流程让 AI 协作真正走向工程化。结合我们真实需求开发中的实践，我们梳理出我们项目推进 Spec coding 落地的几个关键层面：



推进 AI 友好型工程

在讨论 AI 编程效果不佳时，很多人第一反应是模型不行、提示词没写好。但实际用下来，影响最大的往往是工程本身的质量。AI写代码会读取项目上下文——已有代码、模块结构、依赖关系。如果工程本身质量不佳，会把 AI 带偏生成出同样混乱甚至相互矛盾的代码。在人工开发时代，可以凭借个人经验和约束规避，但在AI时代，它们会被直接放大为幻觉和错误。所以，如果你打算认真用 AI 来提效，与其研究各种高级的提示词技巧，不如先回头看看自己的工程，给AI做些针对性的优化。

在这一点上，输入法的 Kuikly 跨端项目具备较好的基础。从项目立项之初，到多个平台多个模块的逐步扩展中，我们始终保持较为彻底的架构设计——页面与业务逻辑分离、系统能力统一封装、模块间依赖关系清晰。正因如此，即使在尚未引入任何额外流程，对工程做额外调整的情况下，仅让 AI 参考现有模块生成代码，就已经能有不错的效果。

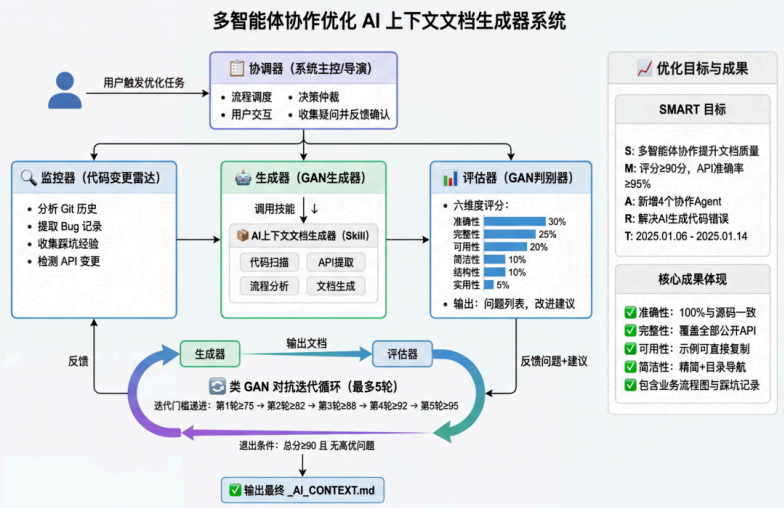


构建精准的AIContext

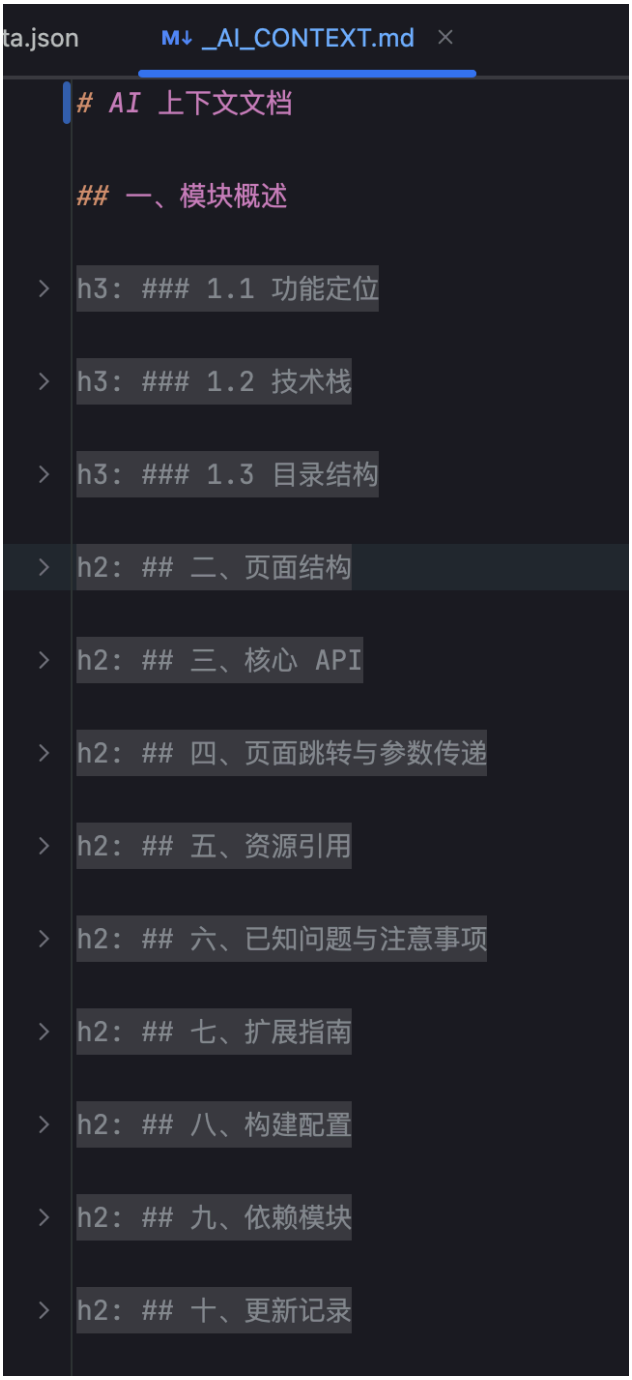
随着模型能力的提升，模型的上下文窗口也变得越来越大，似乎可以将整个项目一股脑塞给AI，但实践中效果可能并非如期。对于开发者来说，哪些系统能力已经封装、哪些基础组件可以复用、哪些服务模块应该优先使用，往往是“默认常识”。

但对于AI，只能在庞杂代码里盲目猜测和搜索，不仅容易漏掉已有能力、重复造轮子，也消耗大量token。另一方面，模型也是基于概率生成输出，输入信息的质量直接影响结果，塞入大量无关代码，AI无法自行判断重要性，输出质量反而下降。

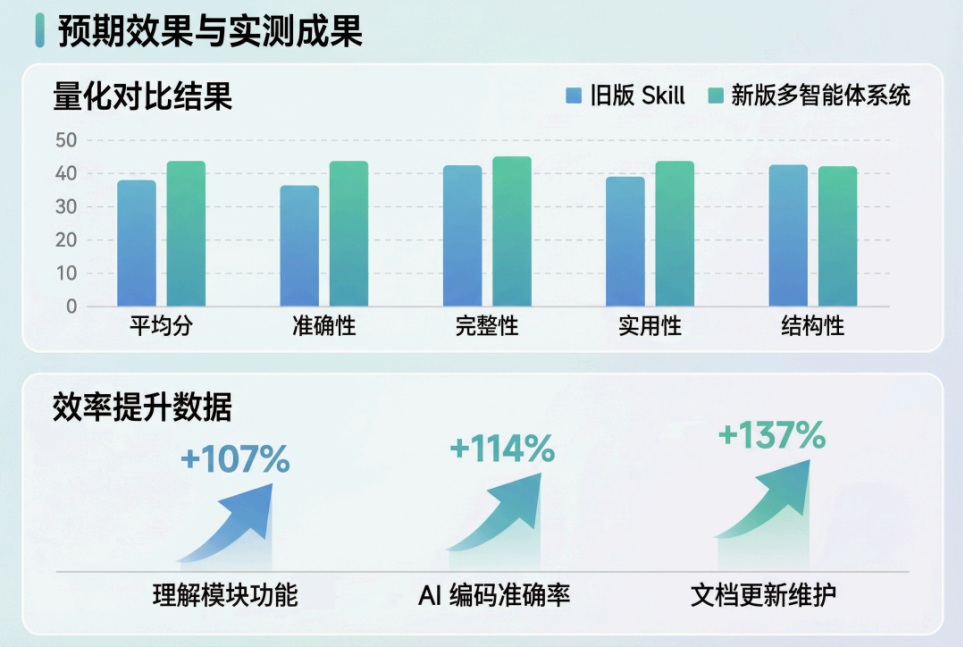
因此，我们构建了AI上下文文档生成器Skills的基础上构建了系统，为各模块生成结构化文档。保留AI需要的关键信息：模块职责、核心 API 、参数含义、模块依赖关系。将这些内容提前沉淀，为 AI 建立对项目的准确理解。同时，这类高质量的 Context 不只是服务于需求当前，在后续模块迭代、修改、扩展时，它们也会成为稳定的“工作记忆”，帮助AI保持实现思路一致、减少偏航，并在多轮协作中持续输出更可靠的结果。



该系统的核心机制借鉴了 GAN（生成对抗网络）的思想——文档并非一次生成即定稿，而是通过生成器与评估器的多轮对抗迭代持续打磨。协调器设定逐轮递增的质量门槛（第 1 轮 ≥75 → 第 2 轮 ≥82 → … → 第 5 轮 ≥95），每一轮由生成器产出文档，评估器打分并给出改进建议，生成器据此修订后再次提交评审，如此往复直至达到质量标准，最终输出高质量的文档，文档从模块职责、核心 API、参数含义、依赖关系等多个维度进行结构化描述，为 AI 提供更精准的上下文支持。



经过验证，通过本系统改造后处理出新的文档在多个指标上取得了提升：质量层面，文档准确性达到与源码完全一致、完整性覆盖全部公开 API、示例代码可直接复制使用。效率层面，新人上手项目提速 50%，理解模块功能提速 107%，AI 编码准确率提升 114%，文档更新维护效率提升 137%，真正实现了"让 AI 更懂工程、让开发更高效"的目标。



标准化需求流程

工程化的一个特点是流程，在没有流程约束的“提示词开发”中，人与AI的协作是点对点、即兴且不可复刻的。

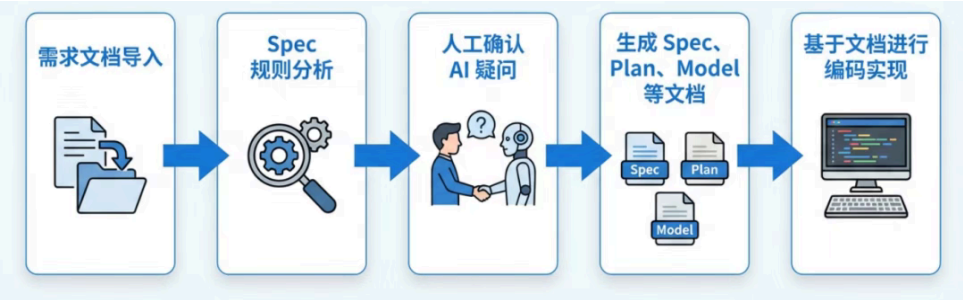
这带来几个问题：

- 质量无法保证：在没有统一流程、缺少明确约束的情况下，产出往往高度依赖两件事：一是当次需求描述是否足够准确，二是 AI 当下的“临场发挥”是否稳定。代码质量波动巨大。

- 知识无法沉淀：个人开发摸索出来提示词，有效经验其实掌握在个人手里，每次开发都重头开始，无法建立一套可复用，可持续的机制。

真正的工程化目标，并不是“让大家都会写提示词”，而是要有一条标准化AI参与流程。只有这样，AI 的能力才能从个人技巧走向工程化的目标。

在流程选择上，我们结合近期的任务形态做了考量：当前大多需求时是新页面开发，而一个客户端页面天然就是边界清晰、目标明确、适合独立交付的微型项目。因此我们最终采用了Spec-Kit，它能很好地把 AI 协作纳入标准流程，让开发从“提示词即兴发挥”变成“基于明确规格的稳定执行”，关于 Spec-Kit 的原理和流程此处不再赘述。效果上，也让Kuikly的同事尝试过，基本可以复刻相同的效果。



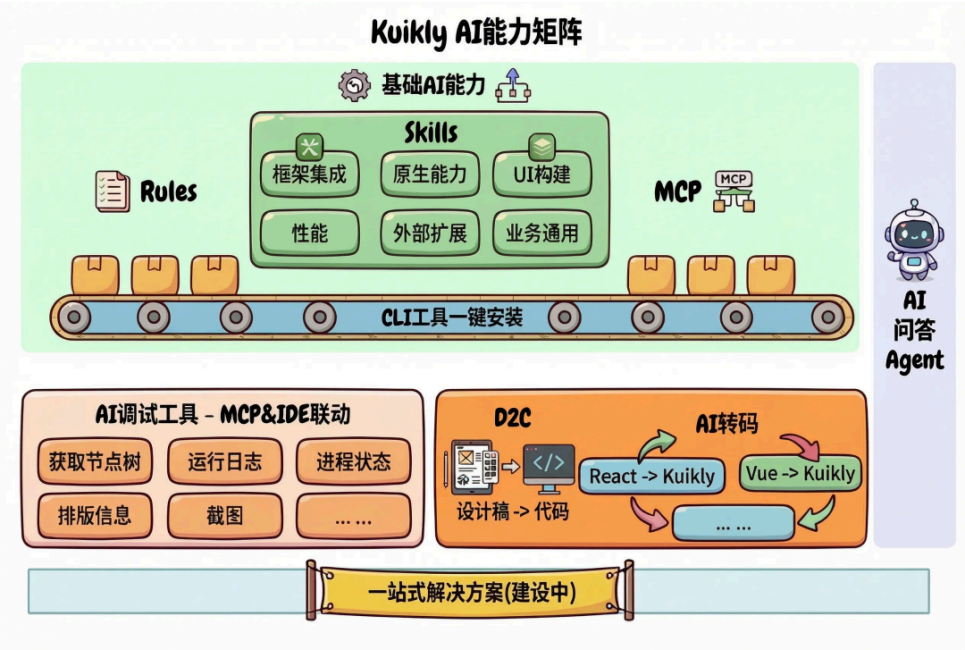
Kuikly开箱即用的AI工具

在初步应用 AI 辅助编程的过程中，我们注意到，往往需要人工介入修改的地方集中在框架规则与实现技能两个层面。有些问题虽然可以通过人工在后续迭代中能够修正，但如果能提前通过工具补齐，整体效率会更高、代码产出也会更稳定。

原因在于，即使AI已经理解了具体的需求、项目上下文，但模型本身的训练语料覆盖仍然有限，对于项目中特定的能力边界和最佳实践认知不完整。针对这类问题，Kuikly 框架已经提供了一整套的 AI 辅助工具**KuiklyAI编程指南**，包括基础的AI能力： MCP 服务、Rules、Skills，也提供了AI调试、D2C和AI转码这类针对特定场景的工具，覆盖了从编码、调试到迁移的多个环节。

目前我们主要使用了 Rules、Skills 和 MCP 这几项核心能力，AI 调试工具、D2C 和 AI 转码等能力也在规划接入中。实际使用下来，即便是直接以 Vibe Coding 的方式，配合这些工具也已经能取得相当不错的生成效果，可以为 AI 补充Kuikly层面的知识，帮助它在生成代码时更准确地理解组件、API 和相关约束条件，从而大幅减少调用幻觉和错误用法的出现。因此，在框架侧的代码生成上，**业务并不需要过多操心模型理解Kuikly框架的能力**，可以将更多精力聚焦在业务逻辑和存量工程的适配上。现在Kuikly也提供了一件配置的CLI工具，Kuikly 团队持续对这些 AI 能力进行维护和迭代更新，当框架侧有新的优化或规则补充时，可以通过 CLI 一键更新即可同步到最新版本，无需手动关注变更细节，实现真正的开箱即用。

同时，我们也结合自身项目在过往开发中积累的实际经验和规范，进一步沉淀了业务使用上的 Rules，明确了架构层级和代码层级的各项约定，从而在更细的粒度上引导和约束 AI 的编码行为。



三、实践与效果

以输入法最新一期灵感词库功能页面开发为例，灵感词库是搜狗输入法键盘端的一个新功能面板，设计稿如下：

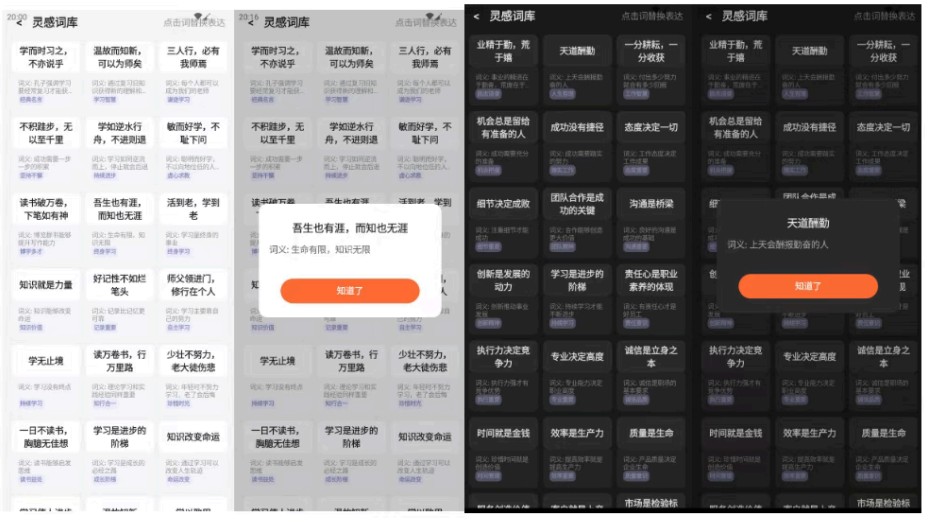


页面细节上涉及动态多列布局适配、多种页面状态管理、暗黑模式切换，同时需要对接网络请求、路由跳转、输入客户端交互、KV 存储、埋点上报等多项服务能力。此外，灵感词库作为一个持续迭代的功能，后续还会有二期需求演进，因此在页面架构设计上也需充分考虑可扩展性，为后续迭代留足空间。

通过 Spec-Kit 的 /speckit.spec → /speckit.plan → /speckit.tasks 三个阶段，将产品需求文档、设计稿和交互稿做需求转化为结构化的工程文档。整个过程由 AI 主导，过程做了在关键节点做确认。最终输出了一套完整的 Spec-Kit 文档体系：



随后进入编码实施阶段，便得到了一个 UI 与功能还原度都较高的版本，效果如下：



模块组织、页面核心框架、组件结构、布局逻辑均无需大改，主要集中在UI细节调整上。

指标	数据
Kotlin 源文件	16 个
代码总行数	~ 2000 行
覆盖功能点	卡片列表、动态布局、词义浮层、骨架屏、错误/空态、暗黑模式
数据模型	3 个
UI 组件	7 个

对比传统开发模式，同等规模的新需求模块页面搭建通常需要 3 天的纯编码和技术方案时间，而借助 AI 工程化流程，1 天即完成了主体开发。同时，得益于 Spec 文档的前置约束和 Rules 的规范引导，生成的代码在架构分层、状态管理、跨端规范等方面都符合项目要求，代码review阶段基本不需要做架构层面的返工。

需要说明的是，上述效果主要体现在**新模块、新页面的场景**中。这类任务边界清晰、依赖可控，AI 的完成度也比较高。而对于存量代码的修改和跨模块的深度重构等场景中，我们也尝试过用同样的流程去推进，但AI 的完成度目前还是因情况而异，稳定性还不够理想，后续还需要持续沉淀各类场景的 Skills，帮助AI更好理解任务。

四、展望

相较于传统 Vibe Coding，当前模式在迭代开发新模块、新页面搭建效率有不错的效果提升。当然，距离 AI 全流程工程化的完整落地仍有一段路要走，我们也在以下几个方向持续探索：

1. 打通 D2C 工具，减少 UI 修正成本

现阶段，还有大量工作集中在 UI 层面的修正与调优上，耗费了较多人力。如果能够打通 D2C 工具链，将设计稿自动转化为高质量的代码产物，从源头上减少 UI 还原过程中的人工介入，提升开发效率。在这一点上Kuikly团队，也推出了视觉稿转码工具，未来我们的流程上也会持续往这方面继续探索。

2.自动化验证

随着整体流程的跑通，仍有一部分环节依赖人工验证。下一步我们计划结合 Kuikly 预览、Inspector 等现有工具能力，逐步构建验证机制。通过 AI 自动化地对 UI 渲染结果、交互行为、布局一致性等进行校验与反馈，逐步替代人工验证环节，实现从"生成→预览→验证→修正"的全流程自动化闭环，进一步提升质量保障效率。

3.扩展更多场景

但实际开发工作远不止于新页面开发，后续我们计划逐步将 AI 的能力延伸到更多场景——包括需求文档的自动解析与任务拆分、线上 BUG 的自动定位与修复、跨端工程与端侧工程联动等，在实践过程持续沉淀更多的 Skills 和工具，将各项能力与研发流程深度编排集成，让 AI 在更多场景中发挥作用，逐步覆盖到最终的完整交付链路。

五、关于Kuikly

当前Kuikly已经开源，有兴趣和有需要的产品，可以通过以下方式访问 Kuikly 仓库和文档，欢迎Star、Watch与体验：

👉 Github 仓库： | 📚 官方文档

Kuikly框架属于腾讯端服务联盟（tds.qq.com）的重要成员，欢迎关注及了解更多信息：

- 腾讯端服务官网： https://tds.qq.com/
- TDS Framework官网： https://framework.tds.qq.com/

「腾讯端服务」服务号	「TDS Framework」公众号	在线咨询



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 5046 修改于2026年3月28日

留言

写留言

还没有留言，来写首评